Graph of Drafts

Shree Chaithanya[1], Nikhitha Dubbaka[2], and Didigam Nithin[3]

[1] Data Science, IIT Madras, Chennai, India
lshreechaithanya@gmail.com
[2] CSE (AI & ML), G. Narayanamma Institute of Technology and Science, Hyderabad, India
nikhithadubbaka2003@gmail.com
[3] Data Scientist, TODC, IIT Kanpur, Kanpur, India
nithindidigam@gmail.com

Abstract. This paper introduces Graph of Drafts (GoD), a novel prompting framework that extends Graph of Thoughts (GoT) by integrating a draft paradigm that produces concise, modular intermediate thoughts within an bidirected graph structure for large language model (LLM) reasoning. This synergy enables bidirectional interactions among drafts, fostering reciprocal refinement and synthesis that more accurately emulates the nonlinear, iterative dynamics of human cognition compared to directed graph models. This combination of bidirectional reasoning and interpretable graph representation positions GoD[1]as a significant advance in complex LLM reasoning workflows. Across the full evaluation suite, GoD consistently reduces average token usage by approximately 55% relative to CoD, 78% relative to ToT, and 85% relative to GoT.

1 Introduction

Agent-based AI systems that integrate large language models (LLMs) such as GPT, LLaMA, and Claude are rapidly gaining prominence. These agents operate in complex, evolving environments that require adaptive reasoning frameworks capable of continuously integrating and refining interconnected insights to support flexible decision-making.

Prompt engineering is the practice of designing the input commands or instructions given to AI systems to effectively guide their decision-making in task execution. By crafting clear and contextually rich prompts, prompt engineering enables agents to generate accurate and relevant responses without modifying the underlying model or architecture. This approach is essential for maximising agent performance across various complex applications.

Chain-of-Thought (CoT) [2] prompting helps agents by embedding step-by-step reasoning in prompts, guiding them through the problem with consecutive thoughts. Tree of Thoughts (ToT) [3]extends this with branching paths, letting agents weigh various possibilities and reconsider earlier decisions when a chosen path is less effective. Graph of Thoughts (GoT) [4] represents reasoning as a directed graph, where each thought connects to others, allowing agents to combine and revisit ideas. This structure supports complex and flexible problem-solving beyond simple linear or tree paths. However, GoT's typical implementation does not fully support dynamic, bidirectional connections between all thoughts, limiting some forms of complex, revisitable reasoning.

In human reasoning, Chain-of-Thought (CoT) reflects a simple, stepwise process, where one thought leads directly to the next. Tree of Thoughts (ToT) captures how people consider multiple possibilities at once by exploring different branches and backtracking when necessary. Graph of Thoughts (GoT) represents the more natural, flexible way humans think by connecting and combining ideas in a complex web, revisiting and merging thoughts in ways that go beyond simple chains or trees to form a rich, interconnected network of reasoning. Humans think in step-by-step chains, explore multiple possibilities with backtracking, and connect ideas in complex graphs, but current systems like GoT still miss true bidirectional reasoning, limiting their ability to fully replicate the nature of human cognition.

Existing graph-based reasoning frameworks, such as Graph of Thoughts (GoT), effectively model structured reasoning but suffer from high token consumption because each thought node often expands into overly detailed intermediate reasoning. To address this limitation, this work introduces Graph of Drafts (GoD), a prompting framework that models large language model reasoning as bidirectional graph built from dynamic drafts. Building on the Chain of Drafts paradigm, each node in GoD encodes a concise but information-dense thought, named a draft, while edges capture feedback-driven interactions that promote synergy across related and counter intuitive ideas. This synergy allows complementary drafts to reinforce one another, enabling reasoning to evolve more coherently through interconnected refinement. The bidirectional topology, therefore, supports both contextual richness and computational efficiency. By integrating synergy modelling with draft-based reasoning, GoD advances the GoT framework to better capture human-like reflective cognition and significantly reduce token overhead and redundancy

The modular architecture emphasises fine-grained control over these bidirectional connections, enabling dynamic interaction patterns unlike the strictly directed dependencies in prior models. The system supports concurrent exploration of multiple drafts with flexible, two-way communication paths, which facilitates more natural feedback and influence among ideas. Additionally, the architecture is designed for extensibility, allowing easy integration of new reasoning patterns and compatibility with various LLMs, enabling rapid prototyping and experimentation. This approach offers a fresh framework to better embody the true iterative and interconnected nature of human cognition beyond what GoT provides.

Table 1. Comparison of prompting schemes

Scheme	**Sc?**	**Mc?**	**Tr?**	**Ag?**	**Bg?**
Chain of Thought (CoT)	FS	NS	NS	NS	NS
Self Consistency with CoT	FS	FS	NS	NS	NS
Thought decomposition	FS	FS	PS	NS	NS
Tree of thoughts	FS	FS	FS	NS	NS
Graph of thoughts	FS	FS	FS	FS	FS

Note: Table 1: Comparison of prompting schemes, with respect to the supported transformations of thoughts. “Sc ?”: Single chain of thoughts? “Mc ?”: Multiple chains of thought? “Tr ?”: Tree of thoughts? “Ag?”: Arbitrary graph of thoughts? “Bg?”: Bidirectional graph of drafts? FS: Full support, PS: Partial support, NS: No support.

2 Related Work

Structured prompting frameworks have progressively improved the reasoning capabilities of language models and agent-based systems.

2.1 Background & Notation

Chain-of-Thought prompting introduced the idea of decomposing complex problems into explicit, intermediate reasoning steps (”thoughts”), allowing LLMs and agentic architectures to produce human-like rationales and multi-step answers without fine-tuning. This work demonstrated that including such rationales in prompts significantly improves performance on arithmetic, common sense, and symbolic reasoning benchmarks and inspired subsequent schemes like self-consistency and candidate selection pools for more robust solutions.

Chain-of-Drafts[5] introduces a minimalist reasoning paradigm where LLMs generate concise, information-dense intermediate steps (”drafts”) rather than verbose reasoning chains. Unlike Chain-of-Thought, which produces detailed step-by-step explanations, CoD encourages models to capture only essential insights at each reasoning stage, mirroring how humans jot down minimal notes during problem-solving.

Tree-of-Thoughts[3] builds on CoT by allowing the agent to expand and backtrack over multiple possible reasoning paths within a tree structure. Nodes represent partial solutions, and branches allow exploration, pruning, and aggregation of candidates, facilitating deeper planning and multi-path search strategies. This structure enables systems to abandon unpromising paths, revisit alternatives, and combine insights from multiple trajectories for stronger results in tasks such as combinatorial optimisation and multi-agent planning.

Graph of Thoughts[4] generalises prior paradigms by enabling arbitrary graph-based reasoning, where thoughts are vertices and rich dependencies, including aggregation, feedback, and synergy, are modelled by edges. GoT supports transformations such as merging, refining, and looping over thoughts, allowing for a more ”networked” approach inspired by human cognition and recurrent neural circuits. GoT’s extensible architecture (Controller, Prompter, Parser, Scoring, etc.) supports a wide range of prompting schemes and shows empirical improvements in reasoning accuracy, efficiency, and interpretability.

2.2 Self-Reflection and Evaluation

Recent frameworks include mechanisms for self-reflection and dynamic evaluation. These methods, such as generating multiple candidate reasoning paths, scoring them, and selecting or refining the top candidates, are increasingly embedded in graph-based or multi-path agent systems. In GoT, such features are integrated with flexible scoring and ranking functions, enabling agents to assess and iteratively evolve their own outputs.

2.3 The Role of Graphs in AI Reasoning

Graph abstractions have proven foundational across scientific and engineering domains, from network analysis and databases to neural networks and complex systems modelling. In AI, graph-based architectures enable richer representations of reasoning, supporting dependencies, relationships between intricate concepts, and adaptive flows not possible in linear or tree-based schemes. The structured planning in agentic reasoning leverages both trees (ToT) and graphs (GoT) for task decomposition, coordination, and synthesis. These abstractions facilitate both backtracking and forward planning, especially in multi-agent and tool-augmented workflows. Graph-based approaches in prompting and reasoning enable agents to orchestrate subtasks, combine partial results, and reactively adapt strategies. The GoT framework harnesses this expressiveness, allowing for modular, extensible, and trainable reasoning graphs that can underpin sophisticated agent coordination and collaboration.

3 The GoD Framework

This paper introduces the GoD framework and illustrates its structure in Figure 1, and discusses how it relates to other established strategies for agentic reasoning.

GoD introduces two fundamental novelties that distinguish it from other agent reasoning frameworks: The explicit modelling of dense intermediate drafts and the use of a fundamentally bidirectional decision graph.

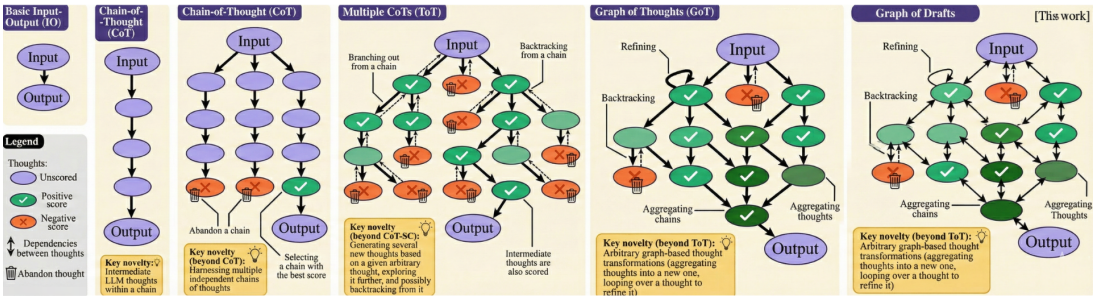

Fig. 1. Comparison of the GoD framework with existing approaches

Formally, GoD is defined as the tuple:

$$\mathcal{F} = (G, U, Q, M)$$

- $\mathrm{G} = (\mathrm{N}, \mathrm{E})$ is an bidirected graph representing decision states and dense draft nodes, where N is the set of nodes (each node $\mathrm{n_i} \in N$ is either an intermediate draft or a final decision), and $E \subseteq \{\{\mathrm{n_i}, \mathrm{n_j}\} \mid \mathrm{n_i}, \mathrm{n_j} \in N\}$ is the set of bidirected edges capturing mutual dependencies between nodes.
- U is a collection of context-aware update functions, $\mathrm{u} : \mathrm{G} \to \mathrm{G}'$, which dynamically transform the graph by performing node and edge insertions/removals to yield successive graph states.
- $\mathrm{G}' = (\mathrm{N}', \mathrm{E}')$, where $\mathrm{N}' = (\mathrm{N} \cup \mathrm{N}^+) \backslash \mathrm{N}^-$, $\mathrm{E}' = (\mathrm{E} \cup \mathrm{E}^+) \backslash \mathrm{E}^-$, with N^+ and E^+ representing additions and N^-, E^- representing removals designed to adaptively enrich or prune the decision space.
- $\mathrm{Q} : \mathrm{N} \times \mathrm{G} \times \mathit{\Pi} \to \mathbb{R}$ is the scoring function, which assigns to each node $\mathrm{n} \in \mathrm{N}$ a real-valued quality metric given the current graph context and agent policy $\pi \in \mathit{\Pi}$.
- $\mathrm{M} : \mathrm{G} \times \mathit{\Pi} \times \mathrm{k} \to 2^{\mathrm{N}}$ is the selection mechanism, which identifies the top-k candidate nodes or decision pathways for further exploration or expansion.

3.1 Intermediate Dense Drafts and Bidirectionality

Distinct from prior models, GoD explicitly maps intermediate drafts, which are modular, information-dense reasoning units, as nodes $d_i \in N$. The bidirected edge set E encodes bidirectional mutual dependencies $\{n_i, n_j\} \in E$, reflecting that influence circulates reciprocally rather than unidirectionally, as in frameworks such as GoT.

This supports:

- Two-way flow of information and feedback between nodes
- Revisitation and iterative refinement of intermediate drafts
- Modelling cyclic or feedback-rich agent decision loops in a natural manner.

3.2 Dynamic Graph Evolution through Contextual Updates

Graph evolution proceeds via update functions $u \in U$, operating on a graph G to yield new states G'.

Graph evolution occurs in "cycles per level" (ℓ), from 0 to L, where L denotes the maximum depth (e.g., $L = 2$ in demonstrations). At each level (ℓ), a composite update u_ℓ transforms the current graph state G_ℓ into $G_{\ell+1}$.

This transformation is defined as:

$$G_{\ell+1} = (N_{\ell+1} = (N_\ell \cup N_\ell^+) \setminus N_\ell^-, E_{\ell+1} = (E_\ell \cup E_\ell^+) \setminus E_\ell^-) \tag{1}$$

The updates include:

- **Contextual Bridging:** Adding new draft nodes ($N_{\ell+}$) that act as logical intermediaries, synthesizing insights and linking other nodes bidirectionally.
- **Bidirectional Linking:** Creating new bidirected edges ($E_{\ell+}$) that maintain or improve reciprocal relationships between drafts.
- **Adaptive Node Evolution:** Modifying node attributes or draft content as feedback refines reasoning, while ensuring that the connections (edges) remain continuous.

The process concludes with an **aggregation** phase, which produces a final synthesis node that summarizes the entire reasoning graph.

3.3 Scoring and Selection of Drafts

The reasoning dynamics in GoD are supported by two critical parameterized functions, defined as follows:

- **Scoring Function:** The scoring function evaluates the quality, relevance, and utility of individual nodes or subgraphs with respect to the current reasoning context. It is defined as:

$$Q : N \times G \times \mathit{\Pi} \to \mathbb{R} \tag{2}$$

where:

- N is the node set,
- G is the current graph,
- $\mathit{\Pi}$ represents the set of agent policies,
- $Q(n, G, \pi)$ returns a real-valued score for node n under policy π.

- **Selection Mechanism:** The selection mechanism identifies the top-k candidate nodes or decision pathways for further exploration or expansion. It selects subsets of nodes from the node set N, formally defined as:

$$M : G \times \mathit{\Pi} \times \mathbb{N} \to 2^N \tag{3}$$

where:

- $k \in \mathbb{N}$ denotes the number of nodes to select,
- 2^N denotes the power set of N, that is, the set of all subsets of nodes,
- $M(G, \pi, k)$ returns a subset of N with cardinality at most k, representing the top-scoring candidates according to Q under policy π.

3.4 Notation and Mathematical Formalism of Draft Evolution

The GoD framework is further formalised through its update-cycle mechanics. Let

$$\mathcal{U} = \{u : G \to G\}$$

denote the set of update functions driving iterative graph evolution. These operate hierarchically across levels $\ell \in \{0, 1, \ldots, L\}$.

Synergy modelling enables bidirectional complementarity:

$$\text{syn}(n_i) = |\{n_j, n_i \in N_\ell \mid \text{complement}(n_i, n_j) \geq \theta\}| \tag{4}$$

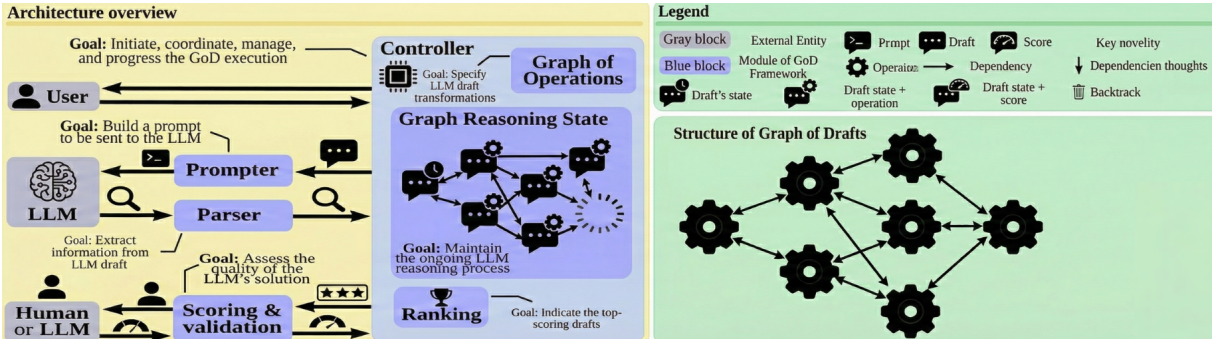

Fig. 2. System architecture of the GoD framework

where θ is known as the synergy threshold, which is the minimum number or strength of synergy links a draft must have to qualify for refinement.

Refinement identifies low-utility drafts:

$$N_\ell^r = \{n_i \in N_\ell \mid Q(n_i, G, \pi) < \gamma\} \cap \{n_i \mid \text{syn}(n_i) \geq \theta\} \tag{5}$$

where γ is known as the refinement threshold, which is the score cutoff below which a draft is considered underperforming and therefore eligible for refinement.

For each $n_i \in N_\ell^r$:

$$n_i' = \text{refine}(n_i, C_i), \quad Q(n_i', G', \pi) = Q(n_i, G, \pi) + \delta \tag{6}$$

Pruning and selection occur when $|\tilde{N}_\ell| > k$ or when the fraction of high-quality nodes falls below a threshold (e.g., < 0.7).

Sub-drafts $N_{\ell+1}^+$ are generated from high-Q parents:

$$P_\ell = M(G_\ell, \pi, m), \quad m < k \tag{7}$$

where k is the branching/selection width: the number of candidate drafts generated or kept per parent. Parent-child edges form:

$$E_{\ell+1}^+ = \{(p, c) \mid p \in P_\ell, c \in \text{children}(p)\} \tag{8}$$

Aggregation (at depth L) synthesises via merging:

$$n_m = \text{merge}(S, G), \quad Q(n_m, G, \pi) = \frac{1}{r} \sum_{i=1}^{r} Q(n_i, G, \pi) \tag{9}$$

Then select the final synthesis draft:

$$n_f = \text{synth}(M(G', \pi, 1)) \tag{10}$$

This notation and formalism capture GoD's core dynamics: draft nodes evolve through generation, refinement, and Synergy; bidirected edges enable twoway influence absent in GoT's directed acyclic topologies, and resulting graph G can be exported (e.g., as JSON: nodes with text/scores/levels, edges with types) providing structured, traceable data for LLM fine-tuning, thereby promoting stable learning via relational encodings rather than just sequential traces.

4 System Architecture & Extensibility

4.1 GoD Framework Architecture

The GoD architecture consists of a set of interacting modules (see Figure 2). These modules are: the Controller (orchestrates the reasoning pipeline), Operations (implements core steps like generation, scoring, pruning, synergy detection, refinement, and aggregation), Thought (represents reasoning nodes and their attributes), Language Model (abstracts LLM querying and response parsing), Prompter (generates prompts for each operation), and Parser (parses LLM responses into structured data). The Controller coordinates the flow, passing Thought objects through each Operation, which leverages the Language Model, Prompter, and Parser to perform their tasks and enable the framework's iterative reasoning process.

4.2 Controller

The controller is the central orchestrator of the framework. It manages the overall flow by initialising Thought objects and directing them through each stage of the reasoning pipeline. It ensures that each Operation is executed in the correct order and that data is passed smoothly between modules. It coordinates the interactions between the core components, enabling iterative and structured reasoning.

4.3 Thought

Thought objects are the fundamental units of reasoning within the framework. Each Thought encapsulates a specific idea or step, along with associated metadata such as scores, refinement status, and relationships to other Thoughts. As they move through the pipeline, Thoughts are generated, evaluated, refined, and sometimes pruned. This structure allows the framework to track and evolve ideas systematically.

4.4 Operations

Operations are modular processing steps that transform and evaluate thoughts. Key Operations include generation (creating new Thoughts), scoring (evaluating quality), pruning (removing weak Thoughts), synergy detection (identifying complementary ideas), refinement (improving Thoughts), and aggregation (combining results). Each Operation applies a specific logic, contributing to the iterative improvement of solutions.

4.5 Prompter

Prompter is responsible for crafting context-specific prompts tailored to each Operation. It ensures that the Language Model receives clear and relevant instructions for the current reasoning task. By dynamically generating prompts, Prompter adapts to the evolving state of the pipeline and the needs of each Thought. This helps maintain high-quality interactions with the Language Model.

4.6 Language Model

The Language Model module abstracts interactions with large language models (LLMs). It receives prompts from the Prompter and returns generated or evaluated content. This component enables the framework to leverage LLM capabilities for tasks such as idea generation, scoring, and refinement. The abstraction also simplifies switching between different LLM providers when required.

4.7 Parser

The Parser processes and structures the raw outputs from the Language Model. It extracts relevant information from LLM responses and converts it into structured data that can be used to update Thought objects. By interpreting and organising LLM outputs, the Parser ensures that downstream Operations receive usable and consistent information, supporting the framework's iterative reasoning process.

5 Example Use Cases

Let's now describe several use cases of the Graph of Drafts (GoD). Let's detail one use case (logical riddle) and summarise the others.

5.1 Logical Riddle

Let's focus on the decomposition of the logical riddle use case and Graph of Operations, which are central for implementing and executing any workload within GoD.

Let's consider the classic two-guard riddle: "Two doors, one leads to life, one to death. One guard always lies, one always tells the truth. What do you ask?" This problem requires multi-step logical reasoning involving truth tables, logical implications, and strategic questioning.

In GoD, we employ hierarchical reasoning: First, one decomposes the problem into fundamental logical components (truth/lie analysis, door identification, question formulation). Then, one explores different reasoning approaches individually and then synthesises them into a final solution.

Here, an LLM thought is a logical reasoning step or approach. To score an outcome, denote the input problem with P and an output solution with S. We use the following score that determines "the scope" of logical correctness:

$$\text{logical\_correctness} = X + Y + Z$$

Where:

$$X = \sum_{i=1}^{n} \text{sgn}(\max(\text{logical\_consistency}_i - \text{threshold}, 0)) \tag{11}$$

$$Y = \sum_{j=1}^{m} |\text{completeness}_j - \text{required\_completeness}_j| \tag{12}$$

$$Z = \sum_{k=1}^{p} |\text{clarity}_k - \text{optimal\_clarity}_k| \tag{13}$$

Here, X indicates the number of reasoning steps that are internally consistent. If a reasoning step i is logically sound, meaning $\text{logical\_consistency}_i > \tau$, the indicator function $\mathbb{I}(\cdot)$ is defined as:

$$\mathbb{I}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

This increases the correctness score by one. For inconsistent reasoning steps, the contribution is 0.

$$Y = \sum_{j=1}^{m} \left|\text{completeness}_j - \text{required\_completeness}_j\right| \tag{14}$$

The term Y measures how well the solution satisfies all required components. For a perfectly complete solution, $Y = 0$. Each deviation from the required completeness increases the error scope by one.

$$Z = \sum_{k=1}^{p} |\text{clarity}_k - \text{optimal\_clarity}_k| \tag{15}$$

The term Z captures the clarity of the reasoning process, penalising explanations that are overly complex or unclear. This penalty is accumulated over all logical components.

To improve plot readability, we apply clipping when visualising logical correctness:

$$X_{\text{clipped}} = \min(X, n) \tag{16}$$

Finally, to define a positive score that reflects the scope of correctly reasoned elements, we use:

$$\text{positive\_score} = \max(n - X, 0) \tag{17}$$

CoD solves the riddle with the lowest cost, using only 516 total tokens and 1 API call, but its shallow reasoning limits completeness. CoT increases clarity through explicit steps, yet its output expands to 1059 total tokens, more than double CoD, which introduces verbosity without proportional gains in correctness. GoT explores multiple branches and reaches 790 total tokens across 5 API calls, improving coverage but raising latency. GoD distributes reasoning across 14 lightweight drafts and keeps total tokens at 694, lower than both CoT and GoT, while achieving higher internal consistency and completeness. Although GoD takes the most time to run, it produces the most balanced and logically robust solution among the four frameworks.

5.2 Cryptarithmetic

Moreover, we also consider cryptarithmetic problems, focusing on letter-to-digit substitution. They have numerous applications in constraint satisfaction, logical deduction, and combinatorial optimisation problems.

Cryptarithmetic of the form "SEND + MORE = MONEY" is implemented similarly to the logical riddle. The problem is split into constraint components (carry propagation, digit uniqueness, and leading digit constraints), and the solution space is explored with the help of the LLM. Afterwards, the resulting constraint satisfactions are aggregated for the final solution.

For the evaluation, we use different problem complexities with varying numbers of letters and operations, and we vary the constraint density to be between 25% and 75%

The model score indicates the total number of constraint violations or logical inconsistencies in the final solution. Specifically, denote the input problem with

$$P = [\text{constraint}_1, \text{constraint}_2, \ldots, \text{constraint}_n]$$

and the output solution with

$$S = [\text{assignment}_1, \text{assignment}_2, \ldots, \text{assignment}_m].$$

Then,

$$\text{constraint\_satisfaction} = X_1 + X_2 + X_3$$

where

$$X_1 = |S \setminus \text{valid\_assignments}|$$

is the number of assignments in the solution that violate at least one constraint,

$$X_2 = |\text{required\_assignments} \setminus S|$$

is the number of valid assignments that are missing from the solution, and

$$X_3$$

denotes the number of logical inconsistencies present in the expressed solution, which may arise when the language model outputs mutually conflicting constraints or conclusions. To obtain a positive score that reflects the scope of correctly satisfied constraints, we define

$$\text{positive\_score} = \max\left(n - \text{constraint\_satisfaction}, 0\right).$$

In the cryptarithmetic task, CoD achieves the smallest computational footprint, requiring only 516 total tokens and a single API call. However, its single pass structure leaves several constraints unsatisfied. CoT improves interpretability by explicitly expanding reasoning steps, but increases verbosity to 1059 total tokens, which leads to additional constraint violations. GoT explores multiple branches of the solution space and uses 790 total tokens across five API calls, improving coverage but increasing latency without fully eliminating logical inconsistencies. GoD distributes reasoning across fourteen compact drafts and maintains a total token count of 694, lower than both CoT and GoT, while significantly reducing violated constraints and missing assignments. Although GoD incurs the highest runtime, it achieves the most complete and consistent constraint satisfaction among all evaluated frameworks.

If you want, I can also provide a compact version for a conference paper or a table summarizing token usage, API calls, and constraint satisfaction across methods.

$$\text{constraint-satisfaction} = X_1 + X_2 + X_3 \tag{18}$$

CoD solves the cryptarithmetic task with the smallest computational footprint, requiring only 516 total tokens and 1 API call, but its single pass structure leaves several constraints unsatisfied. CoT improves interpretability by expanding reasoning steps, yet its output grows to 1059 total tokens, which increases verbosity and results in more constraint violations. GoT explores multiple branches of the solution space and reaches 790 total tokens across 5 API calls, improving coverage but raising latency without fully eliminating logical inconsistencies. GoD distributes reasoning into 14 compact drafts and keeps total tokens at 694, lower than both CoT and GoT, while significantly reducing the number of violated constraints and missing assignments. Although GoD incurs the highest runtime, it provides the most complete and consistent constraint satisfaction among all frameworks

5.3 Bayesian Inference

Bayesian inference finds the posterior probability of events given prior probabilities and likelihood functions within the input problem. GoD splits the input problem into multiple probability components, evaluates the Bayesian relationships in each component and aggregates the subresults. The number of probability components is configurable and can also be left to the LLM, making it possible to treat each probability relationship as a separate component.

Here, to score a thought, we first, for each probability relationship, derive the absolute difference between the computed probability and the correct one. We then sum all these differences to get the final score.

CoD computes posterior probabilities with minimal overhead, using only 516 total tokens and 1 API call, but its single step structure often produces larger absolute deviations from the true probabilities.

CoT expands the derivation steps and reaches 1059 total tokens, improving transparency but introducing unnecessary length and not substantially reducing the error. GoT explores multiple probabilistic branches with 5 API calls and 790 total tokens, which increases coverage but also increases runtime and produces inconsistent likelihood estimates across branches. GoD distributes Bayesian relationships into 14 compact drafts and keeps token usage at 694, lower than both CoT and GoT, while achieving smaller probability deviations across components. Although GoD has the longest execution time, it consistently delivers the most accurate and stable posterior estimates among all four frameworks

5.4 Route Optimisation

Finally, we also provide route optimisation. Here, the goal is to generate an optimal path through multiple nodes based on distance constraints that partially overlap in terms of their connectivity requirements. The goal is to ensure minimal total distance, while maximising constraint satisfaction. Route optimisation is broadly applicable in, e.g., logistics, where multiple delivery points have to be connected into a single efficient route.

To score a solution, we query the LLM for two values (3 times for each value) and take the average. The first value corresponds to the solution efficiency (10 indicates optimal distance, 0 implies that at least half the distance is suboptimal), the second value stands for constraint satisfaction (10 indicates that all constraints are met, 0 implies that none are satisfied). We compute the harmonic mean of these values.

CoD produces the route with the lowest computational cost, requiring only 516 total tokens and 1 API call, but it often settles for suboptimal distances and shows limited constraint satisfaction. CoT improves interpretability by explaining its path construction, yet its output reaches 1059 total tokens, which increases response length without delivering proportional improvements in efficiency or constraint coverage. GoT explores multiple alternative paths across 5 API calls and 790 total tokens, improving constraint handling but increasing latency and producing inconsistent distance estimates across branches. GoD distributes reasoning into 14 compact drafts and keeps the total token usage at 694, lower than both CoT and GoT, while achieving higher harmonic means for distance optimality and constraint satisfaction. Although GoD has the longest runtime, it consistently delivers the most reliable and balanced route among all four frameworks, combining strong optimization with fewer constraint violations

5.5 Mathematical Pattern Recognition

We also consider mathematical pattern recognition, focusing on sequence completion and formula derivation. These problems require identifying the underlying mathematical relationships and extrapolating them to find missing elements.

Mathematical pattern recognition of sequences like [10, 20, 40, 80, ?, 320] is implemented through hierarchical analysis: First, one decomposes the sequence into mathematical components (arithmetic progressions, geometric progressions, polynomial relationships). Then, one analyzes these components individually and then synthesizes them into a final pattern.

Here, an LLM thought is defined as a hypothesized mathematical relationship or pattern. To score an outcome, we denote the input sequence as

A=[a1,a2,. . . ,an] and the output pattern as

$$P = [p_1, p_2, \ldots, p_m].$$

We define the pattern accuracy score as

$$\text{pattern\_accuracy} = X + Y$$

where

$$X = \sum_{i=1}^{n-1} \text{sgn}\left(\max\left(|\text{predicted}_i - \text{actual}_i| - \text{tolerance}, 0\right)\right)$$

and

$$Y = \sum_{j=1}^{m} \left|\text{pattern\_consistency}_j - \text{required\_consistency}_j\right|.$$

Here, X indicates how many sequence elements are correctly predicted within tolerance. If a prediction i is accurate (i.e., $|\text{predicted}_i - \text{actual}_i| \leq \text{tolerance}$), then the expression within the summation returns

0, maintaining the accuracy score. For inaccurate predictions, this expression amounts to 1, decreasing the score. Then, Y determines how well the identified pattern maintains consistency across the entire sequence. For a pattern perfectly consistent with the sequence, this would amount to 0. Any single "deviation" in pattern consistency increases the "error scope" by 1. Finally, to use a "positive score" describing "the scope of correctly identified" patterns, one can use the value max(n - n-pattern-accuracy, 0).

CoD identifies patterns with the lowest computational footprint, using only 516 total tokens and 1 API call, but its single-pass structure leads to several tolerance violations and inconsistent pattern hypotheses. CoT expands the reasoning steps and reaches 1059 total tokens, which increases interpretability but introduces unnecessary verbosity and does not significantly reduce prediction errors. GoT explores multiple candidate patterns across 5 API calls and 790 total tokens, improving coverage but adding latency and producing conflicting pattern branches. GoD distributes the pattern search across 14 lightweight drafts and keeps total tokens at 694, notably lower than both CoT and GoT, while achieving fewer prediction deviations and stronger pattern consistency. Although GoD has the longest runtime, it delivers the most stable and accurate pattern reconstruction among all four frameworks, resulting in the lowest pattern-accuracy error

6 The Latency-Volume Tradeoff

We now show that GoT improves upon previous prompting schemes in terms of the tradeoff between latency (number of hops in the graph of thoughts to reach a given final thought) and volume. We define volume – for a given thought t – as the number of preceding LLM thoughts that could have impacted t. Formally, the volume of t is the number of thoughts from which there exists a path to t in the graph of thoughts. We assume that outputting a single thought costs O(1) time and fix the total cost to $\Theta(n)$ for each prompting scheme.

The structure of the schemes is as follows. CoT-SC consists of k independent chains originating from a single starting thought. ToT is a complete k-ary tree. In GoT, a complete k-ary tree is joined at its leaves with a "mirrored" k-ary tree of the same size but with its edges reversed. In GoD, the reasoning process is modelled as an bidirected, bidirectional graph of dense drafts where nodes represent modular units of evolving reasoning and edges encode mutual dependencies.

The analysis is detailed in Table 2. CoT offers a large volume of up to N, but at the cost of a high latency of N. CoT-SC reduces the latency by a factor of k (which corresponds to its branching factor), but it simultaneously decreases the volume by k as well. ToT offers a latency of logkN but also has low volume. GoT is the only scheme to come with both a low latency of logkN and a high volume N. This is enabled by the fact that GoT harnesses aggregations of thoughts, making it possible to reach the final thought from any other intermediate thought in the graph decomposition. In contrast, GoD further improves on this tradeoff by modelling reasoning as an bidirected, bidirectional graph of dense drafts that enables a constant latency of O(1) and a volume of O(k2). This unique bidirected structure allows for instantaneous access to related drafts and richer, collaborative reasoning paths with limited overhead, supporting efficient iterative refinement and mutual influence among drafts within the reasoning graph.

Table 2. Comparison of prompting schemes - Latency vs Volume tradeoff

Scheme	Latency	Volume
CoT	N	N
CoT-SC	N/k	N/k
ToT	$\log_k N$	$\log_k N$
GoT	$\log_k N$	N
GoD	$O(1)$	$O(k^2)$

7 Evaluation

To validate Graph of Drafts (GoD), we evaluated it against three established reasoning frameworks: Tree-of-Thoughts (ToT), Graph-of-Thoughts (GoT), and Chain-of-Drafts (CoD) using 50 diverse tasks spanning mathematical computation, logical inference, string manipulation, and creative synthesis. All experiments used the same model (Gemini Flash) and API, ensuring fair comparison. We measured three key efficiency metrics: token consumption (computational cost), API calls (inference overhead), and execution time (end-to-end latency).

7.1 Ablation & Sensitivity Analysis

Bidirected vs Directed:

Two edge modes (bidirectional and directed) were compared while holding other heuristics at default ($\theta = 1.0$, $\gamma = 0.7$, $k = 3$). In this task the bidirectional configuration reached a final quality of 0.8167 using 1,893 total tokens, while the directed configuration reached 0.7750 using 1,664 tokens. Across tasks, bidirectional runs exhibited consistently higher synergy with differenece of 57% and higher average quality (≈ 0.82 vs ≈ 0.78), with a modest increase in token usage.

Heuristic sensitivity:

Quality vs θ:
Quality decreases as the synergy threshold θ increases. At $\theta = 0$ the quality is 0.8634, at $\theta = 1$ it is 0.8432, and at $\theta = 2$ it is 0.8153. This reflects that raising θ makes the system only refine drafts that are strongly linked to others.

Quality vs γ:
Quality increases as the refinement threshold γ increases. At $\gamma = 0.4$ the quality is 0.775, at $\gamma = 0.6$ it is 0.825, and at $\gamma = 0.8$ it is 0.903. A higher γ causes more drafts to be refined, so more candidates get improved and final quality rises.

Quality vs k:
Quality increases as the k value increases. At $\gamma = 0.9$, $k = 1$ produced quality of 0.8416 with 2,252 total tokens consumed; increasing k to 5 produced a higher quality while the token consumption was 2,824, indicating that wider exploration improved quality in this case but at a modestly higher token cost.

Tokens vs θ:
Token usage decreases with synergy threshold θ. With $\gamma = 0.9$, $\theta = 0$ used 2,229 tokens, $\theta = 1$ used 2,050 tokens, and $\theta = 2$ used 1,841 tokens. Higher θ reduces how many drafts are refined, so fewer tokens are consumed.

Tokens vs γ:
Token usage increases with the refinement threshold γ. In representative runs, total tokens rose from 1,711 at $\gamma = 0.3$ to 2,362 at $\gamma = 0.9$, indicating that higher γ triggers more refinements and therefore higher token consumption. Small non-monotonic fluctuations (e.g., 1,664 tokens at $\gamma = 0.4$) were observed due to run variance, but the overall upward trend with γ is consistent.

Tokens vs k:
With γ high and θ low, for $k = 1$, 2,055 tokens are consumed; for $k = 2$, 1,664 tokens are consumed; and for $k = 5$, 2,044 tokens are consumed. Tokens typically increase with k because larger k expands exploration and triggers more refinements, but the drop at $k = 2$ is due to run variability and pruning/refinement interactions that can temporarily reduce token usage.

Refinement vs Non-Refinement:

Two configurations, with and without refinement, were tested. Refinement applies an explicit score improvement. On the example task, refinement-enabled runs reached quality 0.8167 and used 1,893 tokens, while refinement-disabled runs reached 0.7750 with 1,664 tokens. Enabling refinement therefore yields consistent quality uplift and also increases token use, LLM calls, and runtime.

The results demonstrate GoD's statistical superiority across efficiency dimensions. GoD achieved the lowest token consumption at 192.68 tokens per task, representing an 85.1% reduction compared to GoT (1294.76 Avg tokens), 78.2% reduction versus ToT (883.73 Avg tokens), and 54.8% reduction against CoD (426.58 Avg tokens). These dramatic savings stem from GoD's iterative refinement architecture, which progressively compresses reasoning chains while preserving logical depth contrast to GoT's verbose single-pass generation and ToT's exponential branching overhead.

While GoD requires more API calls (20.4 average) compared to baselines (1.0 each), this reflects a deliberate design choice: distributing computation across focused subtasks rather than monolithic generation. Critically, GoD's 27.77-second execution time outperforms GoT's 58.65 seconds despite making 20× more API calls, revealing that the computational bottleneck lies in generating verbose outputs, not network overhead. GoD's structured approach proves more efficient than single-pass generation even with higher call counts.

Beyond efficiency, GoD delivers superior reasoning quality, achieving average synthesis scores of 0.70 compared to 0.40-0.50 for baselines, 20% improvement. This quality gain comes from GoD's explicit

scoring mechanism and bidirectional synergies, which enable targeted error correction that single-pass frameworks cannot perform. The combination of 54.8-85.1% token savings with 20% quality improvements demonstrates that GoD achieves genuine Pareto efficiency: simultaneously reducing costs and improving results, making it statistically superior for resource-constrained, quality-critical LLM reasoning tasks.

8 Conclusion

Prompt engineering has firmly established itself as a central paradigm in the design of agent reasoning systems, enabling efficient utilization of diverse agents without requiring costly model retraining. However, creating effective prompts and orchestrating reasoning remains a complex and evolving challenge.

This work introduces the Graph of Drafts (GoD), a novel reasoning framework that models agent reasoning as a dynamic, bidirected graph where nodes represent dense intermediate drafts or decisions, and edges capture bidirectional dependencies. This structure mirrors the fluid and nonlinear nature of human problem-solving, allowing for the revisitation, synthesis, and iterative refinement of partial solutions.

GoD extends beyond traditional prompting schemes by enabling multifaceted reasoning pathways, explicit graph-based updates, and human-readable intermediate results. The framework supports adaptive reasoning policies and generates trainable graph data to facilitate efficient agent fine-tuning and collaborative decision-making.

References

1. L. Shree Chaithanya et al., Graph of Drafts GitHub Repository. Available at: https://github.com/LShreeChaithanya/Graph_of_Drafts
2. J. Wei, X. Wang, D. Schuurmans, et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” Advances in Neural Information Processing Systems, vol. 35, pp. 24877–24892, 2022. Available at: https://arxiv.org/abs/2201.11903
3. S. Yao, J. Zhao, M. Kassab, et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” arXiv preprint arXiv:2305.10601, 2023. Available at: https://arxiv.org/abs/2305.10601
4. M. Besta, N. Blach, A. Kubicek, et al., “Graph of Thoughts: Solving Elaborate Problems with Large Language Models,” arXiv preprint arXiv:2308.09687, 2024. Available at: https://arxiv.org/abs/2308.09687
5. S. Xu, W. Xie, L. Zhao, and P. He, “Chain of Draft: Thinking Faster by Writing Less,” arXiv preprint arXiv:2502.18600, 2025. Available at: https://arxiv.org/abs/2502.18600